

30th Jan 2026 | DA6401

1/1

- Sigmoid Neuron

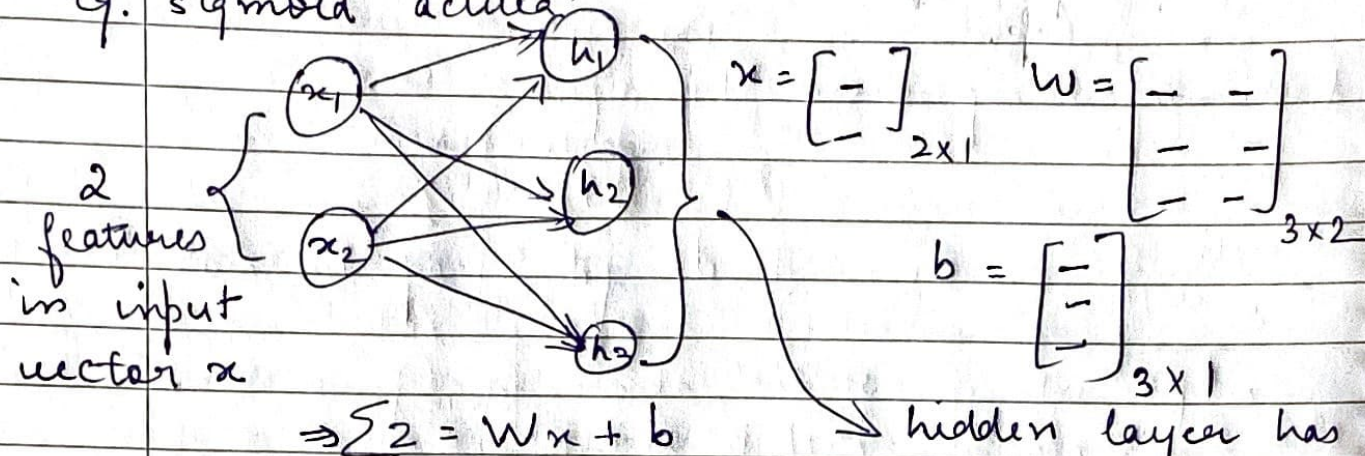
$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$z = W^T x + b$$

smooth or S-curve

$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \rightarrow$ non-zero output $\approx$ in a real value of range $[0, 1]$

eg. sigmoid activation



known as Affine Transformation

$$z = \begin{bmatrix} W \end{bmatrix} \begin{bmatrix} x \end{bmatrix} + \begin{bmatrix} b \end{bmatrix} \quad (\text{hidden activation})$$

these are your learnt stuff

apply non-linearity pt. wise (we apply to every component individually)

$$a = \sigma(z) = \begin{bmatrix} a \end{bmatrix}$$

this is the output of this layer.

This entire process is known as Forward Pass.

- linear model

$$\hat{y} = w_0 x_0 + w_1 x_1 = \sum w_i x_i$$

↳ linear in nets

neural network model

$$a = g(\sum w_i x_i)$$

$a = \underbrace{(\sum)}_{\text{linear op}} \rightarrow g \rightarrow \text{output}$
↳ non-linear opⁿ (w)

in each of them

- if we don't use non-linear activation funcⁿ, the entire thing just collapses into a single linear model. (stacked layers)

↳ this actually allows the model to learn any arbitrarily complex funcⁿ. Helps to learn their decision boundary.

⇒ UAT (Universal Approx. Th.)

$\sigma(\cdot) \rightarrow$ sigmoid

$$\boxed{|F(x) - f(x)| < \epsilon \quad \forall x}$$

$\downarrow$
funcⁿ by model

$\downarrow$
true funcⁿ

- _/_/_
- NN are universal funcⁿ approximators
 - in theory, simple 2-layer networks can solve any problem, iff. we have enough neurons.

- we can control steepness with w (gain factor)
 - small $w \rightarrow$ funcⁿ lazy & linear near margin
 - large $w \rightarrow$ funcⁿ transitions rapidly from 0 to 1.
 - $w \rightarrow \infty \rightarrow$ sigmoid converges to a hard-step.

- control posⁿ by b (bias) $\rightarrow$ acts like a switch.



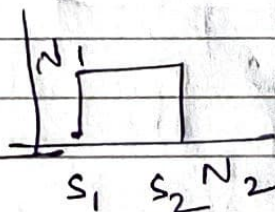
center of the step is shifted to position

$$s = 1 - b/w$$

- if we do $w \rightarrow \infty$ & a specific bias, you can simulate Heaviside step funcⁿ $H(x)$.

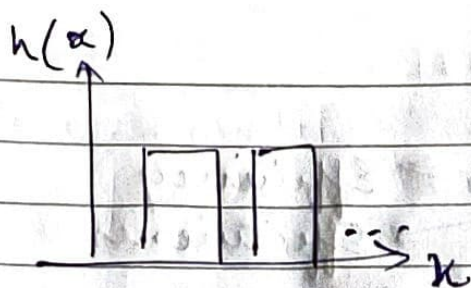
OFF ON

$$\lim_{w \rightarrow \infty} \sigma(w(x-s)) = \begin{cases} 0 & \text{if } x < s \\ 1 & \text{if } x > s \end{cases}$$



rec. funcⁿ (bump) that is non-0 btrue only s_1 & s_2 in the input space

we can have indep. bumps (hidden units) there are all local. It is 0 everywhere except for the specific regions.



- Final $\rightarrow F(x) = \sum_{j=1}^N v_j \cdot h_j(x)$

Role of output nets. $(v_j) \rightarrow$ They scale the height of each bump to match the target funcⁿ $f(x)$ at that locaⁿ.
(Riemann sum)

$N \rightarrow \infty$, approximation error $\rightarrow 0$



- UAT is an existence problem, not a constructive theorem.
- instead of 1 wide layer, we have deeper layers. They represent complex funcⁿs more efficiently (with fewer total params).